

Кодировки

Если вы откроете любую хорошую книжку по криптографии или теории кодирования, то первое, что вы там прочитаете (в предисловии или во введении), будет разница между этими двумя предметами. Очень важно это понимать! Криптография преследует цель сохранения сообщения в секрете, т.е. защищает содержимое сообщения от нежелательных глаз. Теория кодирования в свою очередь пытается ответить на следующий вопрос — как записать сообщение в такой форме, чтобы искажение некоторой части записи (например, при передаче данных) не помешало восстановлению сообщения в исходной форме, т.е. теория кодирования занимается попыткой защитить содержимое сообщения от всяческих помех. Поэтому в криптографии правильно говорить, что сообщение **зашифровали**, а не закодировали!

Подробно касаться вопросов теории кодирования мы не будем. Цель: познакомиться с кодировками, которые довольно часто встречаются при решении CTF задач.

ASCII

ASCII (American Standart Code for Information Interchange) — стандарт представления текстовых символов в цифровом виде. Основная идея: каждому символу — своё число. Изначально ASCII была 7-битной кодировкой.

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

7 бит хватало ровно для англоязычных пользователей, для большинства европейских и нелатинских языков этого было мало. Поэтому стали использовать 8-й бит и появились расширенные таблицы ASCII. коды от 0 до 127 повторяли оригинальный ASCII, от 128 до 255 отводились под новые символы.

[Таблица ASCII](#)

[Задача ASCII с cryptohack.org](#)

Дан массив чисел: [99, 114, 121, 112, 116, 111, 123, 65, 83, 67, 73, 73, 95, 112, 114, 49, 110, 116, 52, 98, 108, 51, 125]

Решение:

Перед нами, очевидно, массив, в котором числа - это коды ASCII символов.

```
a = [99, 114, 121, 112, 116, 111, 123, 65, 83, 67, 73, 73, 95, 112, 114, 49,
110, 116, 52, 98, 108, 51, 125]
print(bytes(a))
```

HEX

При шифровании полученный шифртекст часто состоит из байтов, которые не являются печатными символами ASCII. Чтобы иметь возможность передавать такие данные, их принято кодировать в более удобный и переносимый между разными системами формат.

Одним из таких методов является шестнадцатеричное кодирование (Hexadecimal). Каждый байт (который можно представить как символ ASCII) преобразуется в свое числовое значение по таблице ASCII, а затем это десятичное число конвертируется в число по основанию 16, то есть в шестнадцатеричный формат. Полученные числа объединяются в одну длинную шестнадцатеричную строку.

Этот способ широко используется в низкоуровневом программировании и компьютерной документации, поскольку в современных компьютерах минимальной адресуемой единицей памяти является 8-битный байт. Значение всего байта удобно записывать двумя шестнадцатеричными цифрами, а значение половины байта (полубайта) — одной цифрой.

[Задача Hex cryptohack.org](https://cryptohack.org)

Дана Нех-строка:

```
63727970746f7b596f755f77696c6c5f62655f776f726b696e675f776974685f6865785f737472696e67735f615f6c6f747d
```

Решение:

```
s =  
'63727970746f7b596f755f77696c6c5f62655f776f726b696e675f776974685f6865785f737472696e67735f615f6c6f747d'  
print(bytes.fromhex(s))
```

Base64

Base64 — стандарт кодирования двоичных данных при помощи только 64 символов ASCII. Алфавит кодирования содержит латинские символы A-Z, a-z, цифры 0-9 (всего 62 знака) и 2 дополнительных символа, зависящих от системы реализации. Каждые 3 исходных байта кодируются четырьмя символами.

Base64 наиболее часто используется в Интернете, поэтому двоичные данные, такие как изображения, могут быть легко включены в файлы HTML или CSS.

[Задача Base64 с cryptohack.org](https://cryptohack.org)

Дана Нех-строка 72bca9b68fc16ac7beeb8f849dca1d8a783e8acf9679bf9269f7bf , необходимо декодировать её в байты, а затем полученные байты закодировать в Base64.

Решение:

```
import base64
s = '72bca9b68fc16ac7beeb8f849dca1d8a783e8acf9679bf9269f7bf'
print(base64.b64encode(bytes.fromhex(s)))
```

Байты и числа

Современные криптографические системы работают с числами, а сообщения состоят из символов. Значит нужно как-то переводить сообщения в числа.

Чаще всего это делают в несколько шагов:

1. Каждую букву сообщения заменяют на соответствующее ей число согласно стандартной таблице ASCII.
2. Эти числа переводят в шестнадцатеричную систему счисления.
3. Все шестнадцатеричные числа объединяют в одну длинную строку. Эту строку можно рассматривать как одно большое число, записанное в шестнадцатеричной системе.
4. При необходимости это большое число можно перевести в нашу обычную, десятичную систему.

Простой пример:

```
Сообщение: HELLO
Буквы в числа ASCII: H=72, E=69, L=76, L=76, O=79
Числа в HEX: 72 = 0x48, 69 = 0x45, 76 = 0x4C, 76 = 0x4C, 79 = 0x4F
Объединяем в одну строку: 0x48454C4C4F
Переводим в обычное число: 310400273487
```

[Задача Bytes and Big Integers c cryptohack.org](https://cryptohack.org/bytes-and-big-integers/)

Дано число

11515195063862318899931685488813747395775516287289682636499965282714637259206269 ,
нужно перевести его в байты.

Решение:

```
from Crypto.Util.number import *

m =
115151950638623188999316854888137473957755162872896826364999652827146372592062
69

print(long_to_bytes(m))
```

Или

```
m =
115151950638623188999316854888137473957755162872896826364999652827146372592062
69
hm = hex(m)[2:]
s = b''
for i in range(0, len(hm), 2):
    s += bytes.fromhex(hm[i:i+2])
print(s)
```

Или

```
m =
115151950638623188999316854888137473957755162872896826364999652827146372592062
69
hm = hex(m)[2:]
print(bytes.fromhex(hm))
```

Исторические шифры

Шифр Цезаря

Знаменитый шифр, названный в честь римского полководца императора Гая Юлия Цезаря. Основная идея: каждая символ открытого текста заменяется символом, находящимся на некотором постоянном числе k позиций левее или правее него в алфавите. Число k — ключ этого шифра. Понятно, что этот шифр ненадёжен — можно просто перебрать все возможные значения для k .

Математическая модель шифра:

Пусть $A = \{x_0, \dots, x_n\}$ конечный алфавит, $0 \leq k < n$ - целое число. $m = x_{i_1}x_{i_2} \dots x_{i_s}$ слово в алфавите A . Тогда шифртекст равен

$$ct = E_k(m) = x_{i_1+k \pmod n} x_{i_2+k \pmod n} \dots x_{i_s+k \pmod n}$$

Очевидно, как получить исходное сообщение

$$m = D_k(ct) = y_{i_1-k \pmod n} y_{i_2-k \pmod n} \dots y_{i_s-k \pmod n}, \text{ где } y_{i_j} = x_{i_j+k \pmod n}$$

[Задача Брутфорс с DUCKERZ](#)

Шифр простой замены

Основная идея шифра: каждой символу алфавита открытого текста ставится в соответствие один символ алфавита шифртекста (этот алфавит может совпадать с алфавитом открытого текста). Соответственно, для шифрования сообщения m нужно заменить каждый символ сообщения по установленному правилу. Ключ шифра - отображение алфавита открытого текста на алфавит шифртекста (или правило, по которому задается это отображение). При замене сохраняется вероятность появления отдельных символов, т.е. если символ $x_i \mapsto y_j$ и символ x_i встречается в исходном тексте n раз, то и в шифртексте символ y_j встретится n раз. Таким образом, для вскрытия такого шифра нужно использовать *частотный анализ*.

[Задача substitution0 с PicoCTF](#)

Шифр Вижинера

Шифр Вижинера по сути представляет собой последовательность нескольких шифров Цезаря с различными ключами.

Сам шифр удобно задавать табличкой:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Ключом шифра является произвольное слово, буквы которого повторяются до тех пор, пока общая длина не будет равна длине открытого текста. Например, если открытый текст JOINMONKECLAN, а в качестве основы для ключа взять слово BANANA, то ключом будет слово BANANABANANAB.

Шифрование происходит следующим образом: в верхней строке таблицы находим символ x_{i_j} , в первом столбце находим символ ключа k_j и берём символ, который находится на пересечении этой строки и столбца. На нашем примере получится, что J перейдет в K, O в O, I в V и так далее. В итоге получим KOVNZ00KRCYAO.

Для взлома шифра обычный криптоанализ уже не подойдет, так как одна буква открытого текста может быть зашифрована в разные символы. Для успешного взлома необходимо определить длину ключа. Для этого можно воспользоваться *индексом совпадений*.

Индекс совпадений — число, которое характеризует вероятность того, что два произвольных символа в строке совпадут. Если f_i — количество i -го символа в строке m длины n , то индекс совпадений считается по формуле:

$$I(m) = \sum_{i=1}^n \frac{f_i(f_i - 1)}{n(n - 1)}$$

Для английского языка индекс совпадений имеет значение 0.0667. Для текста на английском языке зашифрованного с помощью шифра простой замены этот индекс также равен 0.0667, потому что количество различных букв в тексте остается неизменным.

Именно это свойство используется для нахождения длины ключа шифра Вижинера. Это свойство используется для нахождения длины ключа шифра Вижинера. Из шифртекста по очереди выбираются каждая вторая буквы и для полученного текста считается индекс совпадений. Если результат примерно соответствует индексу совпадений естественного языка, значит длина ключа равна двум. В противном случае из шифртекста выбирается каждая третья буква и опять считается индекс совпадений. Процесс повторяется пока высокое значение индекса совпадений не укажет на длину ключа.

Успешность метода объясняется тем, что если длина ключа угадана верно, то выбранные буквы образуют шифртекст, зашифрованный простым шифром Цезаря. И индекс совпадений должен быть приблизительно соответствовать индексу совпадений естественного языка. После того как длина ключа будет найдена взлом сводится к вскрытию нескольких шифров Цезаря.

[Задача Vigenere с PicoCTF](#)

[Задача KeyForSolve с TaipanByteCTF](#)

Одноразовый блокнот

Идея этого шифра очень проста. Пусть открытый текст $P = b_1b_2 \dots b_n$, где $b_i \in \{0, 1\}$. Ключ $k = k_1k_2 \dots k_n$, где $k_i \in \{0, 1\}$. Тогда шифртекст C вычисляется следующим образом

$$C = P \oplus K = c_1c_2 \dots c_n,$$

где $c_i = b_i \oplus k_i$ ($\oplus$ - сложение по модулю 2, также известное как XOR).

Для дешифрования нужно, очевидно, к шифртексту C прибавить K .

Название шифра наводит на некоторые мысли. А именно: *каждый ключ K следует использовать только один раз*. Если бы один и тот же ключ K использовался для шифрования P_1 и P_2 в C_1 и C_2 , то пассивный противник мог бы подслушать и вычислить

выражение:

$$C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2$$

Тогда противник узнал бы результат XOR P_1 и P_2 , что не есть хорошо. Более того, если противнику известно хотя бы одно из сообщений, то он бы смог узнать и второе.

Одноразовый блокнот не очень удобно использовать, потому что требуется ключ такой же длины как и открытый текст, да и для каждого нового сообщения нужен новый ключ.

[Задача Two Time Pad с DUCKERZ](#)